DAY 1 · MODULE 6 · 35 MIN

Hosting Agent Framework agents

Foundry-hosted and self-hosted — pick what your app needs to own

Where the agent process lives

First choose who operates the infrastructure. Hosting model is separate from the protocol clients use to reach your agent.

Foundry-hosted

Microsoft-managed · GA

Prompt agent

Configuration only — no code

Hosted agent

Your MAF code, containerized · Foundry runs it

Self-hosted

You run the process · Python prerelease

Your app + hosting packages

agent-framework-hosting + protocol packages (Responses / A2A / MCP / Telegram)

Your app owns routing, auth, storage, deployment, scaling

Foundry-hosted family

Microsoft-managed hosting. Generally available today.

- What Foundry runs: the container, autoscale, session persistence, platform integration
- What you own: agent code (Hosted flavor) or configuration (Prompt flavor), plus Foundry settings
- Choose Foundry-hosted when: you want Microsoft-managed hosting and don't need application-level control over the runtime

Flavor	What's inside	Best for
Prompt agent	Configuration only — instructions, model, tools. Versioned.	Fast start, internal tools, no custom orchestration
Hosted agent	Your MAF code, packaged as a container (or zip, Foundry builds the image)	Agents with custom code — with managed hosting

Foundry-hosted · Prompt agent

A Prompt agent is defined entirely as configuration: instructions, model, tools. Author via the SDK / REST (IaC-first norm, CI/CD-friendly), as a declarative YAML definition, or in the Foundry portal (fine for exploration). Foundry runs it — no application code to maintain, no compute to manage.

```
from agent_framework.foundry import FoundryAgent
from azure.identity import AzureCliCredential

agent = FoundryAgent(
    project_endpoint="https://<foundry-
resource>.services.ai.azure.com/api/projects/<your-project>",
    agent_name="docs-assistant",
    agent_version="1.0",
    credential=AzureCliCredential(),
)
result = await agent.run("What is Foundry IQ?")
```

Best for: fast start, internal tools, production agents that don't need custom orchestration.

Foundry-hosted · Hosted agent

Your MAF code (or LangGraph, or the OpenAI / Anthropic Agents SDK), packaged as a container or a source zip. Foundry runs the container with managed endpoint, autoscale, dedicated Entra identity, end-to-end observability, and content safety. Author-time package: agent-framework-foundry-hosting (prerelease). Exposes your agent via the Foundry Responses or Invocations protocol.

```
# From any client — including another agent — connect by name:
from agent_framework.foundry import FoundryAgent

agent = FoundryAgent(
    project_endpoint="https://<foundry-resource>.services.ai.azure.com/api/projects/<your-
project>",
    agent_name="docs-assistant-hosted",
    credential=AzureCliCredential(),
)
```

Best for: agents that call into your own custom code, custom orchestration, and any scenario where you want Foundry to handle hosting, scaling, and identity.

What Foundry manages for you

What you write

- Agent logic (MAF or other frameworks)
- Instructions and tool wiring
- Any custom business code
- Your unit tests and CI
- That's it.

What Foundry gives you

- Managed endpoint (a stable URL)
- Autoscale — container instances per session and request volume
- Dedicated Microsoft Entra identity per agent
- End-to-end tracing — every model call, tool invocation, decision; App Insights integration built in
- Content safety and prompt-injection mitigation
- Foundry Toolbox tools (web search, code interpreter, MCP servers, …)
- Managed conversations / memory (BYO also supported)

Self-hosted family

You run the agent process in your own web app, container, service, or runtime. Your application owns routing, identity, authorization, request policy, storage, deployment, scaling.

Agent Framework provides hosting helpers, not a server:

- Python: agent-framework-hosting (session state) plus protocol packages (-hosting-responses, -hosting-a2a, -hosting-mcp, -hosting-telegram). Prerelease.
- C#: Microsoft.Agents.AI.Hosting (session store, DI integration) plus protocol packages. Prerelease.
- What MAF gives you: AgentState / SessionStore (Python) or AddAIAgent / AgentSessionStore (C#), plus protocol integrations
- Your app plugs these into its own framework (FastAPI, ASP.NET Core, Django, Azure Functions, …)

Choose self-hosted when: you need application-level control or must integrate with existing infrastructure.

Self-hosted · Python — Agent + Responses API

The simplest self-hosted form: your app calls agent.run(...) directly. Your process. Your runtime. No protocol endpoint yet.

```
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    name="DocsAssistant",
    instructions="You are a helpful docs assistant. Cite sources.",
)
result = await agent.run("What is Foundry IQ?")
```

Additive to Foundry-hosted — the same MAF code can be repackaged as a Foundry-hosted Hosted agent later. No rewrite.

Self-hosted · C# equivalent

```
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;

AIAgent agent = new AIProjectClient(
        new Uri(endpoint), new DefaultAzureCredential())
    .AsAIAgent(
        model: model,
        name: "DocsAssistant",
        instructions: "You are a helpful docs assistant. Cite sources.");

Console.WriteLine(await agent.RunAsync("What is Foundry IQ?"));
```

Same shape. Same primitives. Same "your process calls Foundry's Responses API" pattern.

Self-hosting: pick a protocol

If clients need to reach your self-hosted agent over the network, add a protocol integration package. Same agent target, different clients.

Protocol	Python package	C# package	Use for
OpenAI Responses / Chat	agent-framework-hosting-responses	Microsoft.Agents.AI.Hosting.OpenAI	Any OpenAI-compatible client
Agent-to-Agent (A2A)	agent-framework-hosting-a2a	(protocol integration)	Discovery + messaging between agents
Model Context Protocol	agent-framework-hosting-mcp	(protocol integration)	Expose agent as an MCP tool
Telegram Bot API	agent-framework-hosting-telegram	—	Native Telegram bot

Rule of thumb: hosting model = who runs it. Protocol = how clients reach it. Pick them separately.

Compare at a glance

Concern	Foundry · Prompt	Foundry · Hosted	Self-hosted
Runtime code to maintain	None	Yours	Yours
Compute to manage	None	Container (Foundry)	Yours
Managed endpoint	Yes	Yes	You provide
Autoscale	Yes	Yes	You handle
Agent identity (Entra)	Yes	Yes, dedicated	You handle
Iteration speed	Portal + publish	Portal upload / redeploy	Edit + restart
Portability off Foundry	Low	Medium	High

Decision guide (rough cuts)

- Getting started, or building a scoped internal tool with no custom logic? → Foundry-hosted · Prompt agent
- Shipping a production agent with custom code that needs managed hosting, Entra identity, observability? → Foundry-hosted · Hosted agent
- Regulated agent that needs managed content safety, a stable endpoint, and dedicated identity? → either Foundry-hosted flavor
- Embedding an agent inside an existing app you already run somewhere? → Self-hosted
- Prototyping quickly on your laptop before you decide on hosting? → Self-hosted
- Need to integrate with existing infrastructure — auth, tenancy, storage — that you already control? → Self-hosted

Mix and match — real systems combine both families.

Common gotchas

- "Prompt agent = client-side" — wrong. A Prompt agent is Foundry-managed. There's no client-side runtime for it at all.
- "Hosted agent = my code running anywhere in Azure" — wrong. Hosted agent specifically means your code as a container run by Foundry Agent Service. Your own container in your App Service that calls Responses is self-hosted, not Foundry-hosted.
- "Self-hosted means no MAF hosting packages" — wrong. Self-hosting is where the agent-framework-hosting-* packages live.
- Assuming portability off Foundry — Prompt is Foundry-only by construction. Hosted keeps Foundry-managed features behind the endpoint. Self-hosted is the most portable.
- Mixing hosting model with protocol — hosting model = who runs it. Protocol = how clients reach it. Both families expose Responses.

Same MAF code, different destinations

Self-hosted MAF code can be repackaged as a Foundry-hosted Hosted agent later. The MAF code you write does not change.

Local dev on your laptop

- uv run from a terminal
- F5 in VS Code
- Fast iteration
- No cloud round-trip for logic

Same MAF code

no rewrite

Foundry Agent Service (Hosted)

zip → portal → Foundry-hosted managed

Azure Container Apps or AKS

Self-hosted — you own the container

App Service or Azure Functions

Self-hosted — event-driven or HTTP

Prototype self-hosted. Decide destination later. Foundry-specific features (Toolbox, IQ, agent identity) become available on Foundry-hosted.

What you'll do in the lab

- Part A — Foundry-hosted Prompt agent: create in your Foundry project, connect from your MAF app
- Part B — Foundry-hosted Hosted agent: deploy your own with azd; connect and walk the portal (endpoint, tracing, dedicated identity, content safety)
- Part C — Self-hosted: build an MAF app in Python that runs in your process and calls Responses
- Stretch (Part C): extend your code with a custom function tool or a Foundry IQ knowledge source (deep dive on Days 2–3)

Same underlying docs-assistant behavior, both hosting families. You'll feel the trade-offs.

Takeaways

- Two hosting families: Foundry-hosted (Microsoft-managed, GA) and self-hosted (your app owns the runtime).
- Under Foundry-hosted: Prompt agents (configuration only) and Hosted agents (your MAF code, containerized).
- Same MAF code can move between self-hosted and Foundry-hosted without a rewrite.
- Hosting model and protocol are separate choices — Responses / A2A / MCP / Telegram can layer on top of either family.
- Foundry Agent Service manages more than models — endpoint, identity, observability, Toolbox tools, memory, content safety.

Next: Lab walkthrough and environment check.